

บทที่ 1

พื้นฐานเจเอสพี

(JSP Basic)

เนื้อหา

- การพัฒนา JSP
- ประวัติความเป็นมาของ JSP
- สถานการณ์ปัจจุบันของ JSP
- เทคโนโลยีที่มาแทนที่ JSP
- ส่วนประกอบของ JSP
- ขั้นตอนการตั้งค่าและใช้งาน JSP ใน NetBeans
- ตัวอย่างการใช้งาน JSP แบบละเอียด
- ขั้นตอนการตั้งค่าและใช้งาน JSP ใน NetBeans โดยใช้ GlassFish Web Server
- ตัวอย่างการใช้งาน JSP ใน GlassFish
- รูปแบบการเขียนโค้ดของเจเอสพี
- การใช้คำสั่งของภาษาเจเอสพี
- การใช้งานฟอร์มร่วมกับเจเอสพี

JSP (Java Server Pages) คือเทคโนโลยีจาก Java ที่ใช้ในการพัฒนาเว็บแอปพลิเคชันเพื่อสร้างหน้าเว็บที่สามารถแสดงผลข้อมูลแบบ ใดนามิก ซึ่งเปลี่ยนแปลงได้ตามเวลาหรือสถานะต่าง ๆ โดยทำงานร่วมกับเทคโนโลยีอื่น ๆ เช่น Java Servlets และ JavaBeans เพื่อจัดการและประมวลผลข้อมูลฝั่งเซิร์ฟเวอร์ JSP มักถูกนำมาใช้ในเว็บไซต์ที่ต้องการการประมวลผลข้อมูลจากผู้ใช้งาน เช่น การกรอกฟอร์ม การเชื่อมต่อกับฐานข้อมูล หรือการแสดงผลที่เปลี่ยนแปลงตามข้อมูลจากเซิร์ฟเวอร์ โดยสามารถทำงานร่วมกับ Servlet ได้อย่างมีประสิทธิภาพ: Servlet ทำหน้าที่จัดการการประมวลผลข้อมูล และ JSP ทำหน้าที่ในการแสดงผล JSP ช่วยให้การพัฒนาเว็บแอปพลิเคชันด้วย Java มีความยืดหยุ่นและง่ายขึ้นด้วยการแยกส่วนการประมวลผลข้อมูลออกจากการแสดงผล ทำให้สามารถนำไปรวมกับเทคโนโลยีอื่น ๆ ได้อย่างสะดวก

JSP พัฒนาโดย Sun Microsystems (ปัจจุบันคือส่วนหนึ่งของ Oracle Corporation) เพื่อแก้ไขปัญหาข้อจำกัดของ CGI (Common Gateway Interface) และ SSI (Server-Side Includes) ซึ่งเป็นเทคโนโลยีเดิมที่ใช้ในการสร้างหน้าเว็บแบบใดนามิก แต่ขาดความยืดหยุ่นและไม่เหมาะสมกับการใช้

งานในระยะยาวสำหรับเว็บแอปพลิเคชันที่ซับซ้อน JSP เป็นภาษาสคริปต์ที่ประมวลผลฝั่งเซิร์ฟเวอร์ ซึ่งเป็นการขยายขีดความสามารถของ Java Servlet ให้ทำงานในรูปแบบของภาษาสคริปต์ โดยปกติจะอยู่ในแพ็คเกจ `javax.servlet.jsp` ซึ่งรวบรวมคลาสและเมธอดการทำงานต่าง ๆ รวมถึงแพ็คเกจ `javax.servlet.jsp.tagtest` สำหรับสร้างแท็กได้เอง ส่วนประกอบสำคัญของ JSP ได้แก่ ไจเรกทีฟ (Directive) สำหรับกำหนดค่าการทำงานของ JSP Container เช่น `page`, `include`, `taglib`; แอ็คชัน (Action) สำหรับจัดเตรียมและดำเนินการบางอย่างในหน้าเพจโดยใช้แท็กที่กำหนดไว้ล่วงหน้า เช่น `<jsp:include>`, `<jsp:forward>` หรือการเรียกใช้ `JavaBean`; สคริปต์เล็ต (Scriptlet) ซึ่งเป็นส่วนของการเขียนโค้ดภาษา Java ภายใน JSP; และ แท็กไลบรารี (Tag Libraries) ที่ช่วยขยายความสามารถของภาษาโดยการกำหนดและเรียกใช้แท็กที่สร้างขึ้นเองได้

การพัฒนา JSP

1. จุดเริ่มต้นของ JSP

ในช่วงปลายปี 1990s **Sun Microsystems** ได้เห็นปัญหาของการใช้ CGI และ SSI ที่ไม่ได้มีความยืดหยุ่นและมีประสิทธิภาพต่ำ เมื่อเทียบกับเทคโนโลยีใหม่ๆ จึงได้พัฒนา **JSP** ขึ้นมาเพื่อตอบโจทย์ดังกล่าว โดยมีเป้าหมายในการพัฒนาเทคโนโลยีสำหรับการสร้างหน้าเว็บที่สามารถผสมผสานการเขียนโค้ด Java เข้ากับ HTML ได้อย่างง่ายดาย

2. การเปิดตัว JSP

JSP ได้รับการเปิดตัวครั้งแรกในปี **1999** ในเวอร์ชันแรกที่เป็นส่วนหนึ่งของ **Java 2 Enterprise Edition (J2EE)** ซึ่งเป็นสเปคที่ถูกออกแบบมาเพื่อให้สามารถพัฒนาแอปพลิเคชันด้านธุรกิจได้อย่างมีประสิทธิภาพ โดย JSP ถูกใช้ร่วมกับ **Java Servlets** ซึ่งเป็นเทคโนโลยีสำหรับการประมวลผลข้อมูลที่ทำงานบนเซิร์ฟเวอร์

3. การพัฒนาและการขยายตัวของ JSP

หลังจากการเปิดตัวครั้งแรก JSP ก็ได้รับความนิยมจากนักพัฒนาที่ใช้ Java ในการพัฒนาเว็บแอปพลิเคชัน เพราะมันช่วยให้การพัฒนาเว็บแอปพลิเคชันง่ายขึ้น โดยสามารถแยกแยะระหว่างการแสดงผล HTML และการประมวลผลข้อมูลด้วย Java ได้อย่างชัดเจน

ในปี 2001 JSP 1.2 ถูกปล่อยออกมาและมีการปรับปรุงหลายประการ โดยการเพิ่มการสนับสนุนการใช้ **JavaBeans** และ **custom tags** ที่ช่วยให้สามารถใช้ JSP ในการพัฒนาเว็บแอปพลิเคชันได้อย่างยืดหยุ่นมากขึ้น

4. การพัฒนาและการสนับสนุน JSTL

ในปี 2002 ได้มีการเปิดตัว **JSTL (JavaServer Pages Standard Tag Library)** ซึ่งเป็นการนำเสนอชุดของแท็กมาตรฐานที่ช่วยในการทำงานกับข้อมูล เช่น การวนลูป การจัดการเงื่อนไขต่างๆ และการแสดงผลข้อมูลที่ได้จากฐานข้อมูล โดยทำให้การเขียนโค้ด JSP ง่ายขึ้นมาก เนื่องจากไม่จำเป็นต้องใช้สคริปต์ Java มากมาย

5. การเปลี่ยนแปลงใน Java EE และการรวมกับเทคโนโลยีอื่น ๆ

JSP เป็นส่วนหนึ่งของ **Java EE** (Enterprise Edition) ซึ่งได้รับการพัฒนาอย่างต่อเนื่อง โดยในปี 2006 Java EE 5 ได้รวมเอา JSP และเทคโนโลยีที่เกี่ยวข้อง เช่น **EJB (Enterprise JavaBeans)** และ **JPA (Java Persistence API)** มาพัฒนาร่วมกัน ทำให้สามารถสร้างแอปพลิเคชันทางธุรกิจที่มีความซับซ้อนได้มากขึ้น ในปี 2009 Java EE 6 ได้ทำการปรับปรุงเทคโนโลยี JSP โดยการทำให้สามารถใช้ **annotations** ในการกำหนดค่าและลดความซับซ้อนในการเขียนโค้ด

6. การพัฒนาในยุคปัจจุบัน

ถึงแม้ว่า JSP จะได้รับความนิยมในช่วงแรกๆ แต่ในปัจจุบันได้มีเทคโนโลยีใหม่ๆ ที่มีประสิทธิภาพมากขึ้น เช่น **JavaServer Faces (JSF)**, **Spring MVC**, และ **Thymeleaf** ที่ใช้ในการพัฒนาเว็บแอปพลิเคชันในระบบที่มีความซับซ้อนสูงขึ้น โดย JSP ยังคงใช้ในบางกรณีที่ต้องการความยืดหยุ่นในการพัฒนาและแสดงผล HTML ที่เปลี่ยนแปลงได้

จุดเด่นของ JSP

- แยกการแสดงผลกับการประมวลผล: ทำให้สามารถจัดการการแสดงผลและการประมวลผลข้อมูลได้แยกกันอย่างชัดเจน
- ง่ายต่อการใช้งาน: สามารถเขียนโค้ด Java ร่วมกับ HTML ได้โดยตรง
- สามารถขยายการทำงานด้วย **custom tags** และ **JSTL**: ทำให้สามารถสร้างโค้ดที่สามารถนำกลับมาใช้ใหม่ได้
- รองรับการทำงานกับ **JavaBeans** และฐานข้อมูล: ทำให้สามารถสร้างเว็บแอปพลิเคชันที่มีฟังก์ชันเชื่อมต่อกับฐานข้อมูลได้

JSP เป็นเทคโนโลยีที่เกิดขึ้นเพื่อแก้ปัญหของการพัฒนาเว็บแอปพลิเคชันในยุคก่อน โดยช่วยให้การสร้างหน้าเว็บที่มีการประมวลผลข้อมูลบนเซิร์ฟเวอร์ทำได้ง่ายขึ้น และมีการพัฒนามาอย่างต่อเนื่องจนถึงปัจจุบัน ถึงแม้จะมีเทคโนโลยีใหม่ๆ เข้ามาแทนที่ในบางกรณี แต่ JSP ยังคงเป็นเครื่องมือที่สำคัญในโลกของการพัฒนาเว็บแอปพลิเคชันด้วย Java JSP (JavaServer Pages) เป็นเทคโนโลยีที่พัฒนาโดย **Sun Microsystems** (ปัจจุบันเป็นส่วนหนึ่งของ Oracle Corporation) เพื่อเพิ่มประสิทธิภาพในการสร้างหน้าเว็บที่มีการประมวลผลข้อมูลบนเซิร์ฟเวอร์ โดยการผสมผสานโค้ด Java เข้ากับ HTML ทำให้สามารถสร้างเว็บเพจที่มีความยืดหยุ่นและตอบสนองต่อผู้ใช้ได้ดีขึ้น

ประวัติความเป็นมาของ JSP

- ปี 1999: Sun Microsystems เปิดตัว JSP ครั้งแรกในฐานะส่วนหนึ่งของ **Java 2 Enterprise Edition (J2EE)** ซึ่งเป็นสเปคที่ออกแบบมาเพื่อพัฒนาแอปพลิเคชันทางธุรกิจ
- ปี 2001: JSP 1.2 ถูกปล่อยออกมา โดยมีการปรับปรุงหลายประการ เช่น การเพิ่มการสนับสนุนการใช้ **JavaBeans** และ **custom tags** ที่ช่วยให้การพัฒนาเว็บแอปพลิเคชันมีความยืดหยุ่นมากขึ้น

- ปี 2002: เปิดตัว **JSTL (JavaServer Pages Standard Tag Library)** ซึ่งเป็นชุดของแท็กมาตรฐานที่ช่วยในการจัดการข้อมูล เช่น การวนลูป การจัดการเงื่อนไข และการแสดงผลข้อมูลจากฐานข้อมูล ทำให้การเขียนโค้ด JSP ง่ายขึ้น
- ปี 2006: Java EE 5 รวมเอา JSP และเทคโนโลยีที่เกี่ยวข้อง เช่น **EJB (Enterprise JavaBeans)** และ **JPA (Java Persistence API)** มาพัฒนาร่วมกัน ทำให้สามารถสร้างแอปพลิเคชันทางธุรกิจที่มีความซับซ้อนได้มากขึ้น
- ปี 2009: Java EE 6 ได้ปรับปรุงเทคโนโลยี JSP โดยการทำให้สามารถใช้ **annotations** ในการกำหนดค่าและลดความซับซ้อนในการเขียนโค้ด

JSP ยังคงเป็นเครื่องมือที่สำคัญในการพัฒนาเว็บแอปพลิเคชันด้วย Java แม้ว่าในปัจจุบันจะมีเทคโนโลยีใหม่ๆ เข้ามาแทนที่ในบางกรณี แต่ JSP ยังคงมีบทบาทสำคัญในวงการพัฒนาเว็บแอปพลิเคชัน ในปัจจุบัน, **JSP (JavaServer Pages)** ยังเป็นเครื่องมือที่มีความสำคัญในบางกรณี แต่ก็ไม่ได้เป็นเทคโนโลยีหลักสำหรับการพัฒนาเว็บแอปพลิเคชันใน Java เนื่องจากการพัฒนาเทคโนโลยีใหม่ๆ ที่มุ่งเน้นไปที่การพัฒนาเว็บแอปพลิเคชันในลักษณะที่มีประสิทธิภาพสูงขึ้นและง่ายต่อการใช้งานมากกว่า JSP

สถานการณ์ปัจจุบันของ JSP

1. ลดความนิยมในปัจจุบัน

ถึงแม้ว่า JSP จะเป็นเครื่องมือที่มีประสิทธิภาพในช่วงแรกๆ แต่ในปัจจุบันมีเทคโนโลยีใหม่ๆ ที่มีความยืดหยุ่นมากขึ้น เช่น **JavaServer Faces (JSF)**, **Spring MVC**, และ **Thymeleaf** ที่ได้รับความนิยมมากขึ้น เนื่องจากช่วยให้การพัฒนาเว็บแอปพลิเคชันเป็นไปได้ง่ายกว่า โดยไม่ต้องฝังโค้ด Java ในหน้า HTML โดยตรง

2. การแทนที่ด้วยเทคโนโลยีใหม่ๆ

เทคโนโลยีอย่าง **Spring MVC** และ **Thymeleaf** ได้กลายเป็นทางเลือกยอดนิยมในการพัฒนาเว็บแอปพลิเคชันในสภาพแวดล้อมของ Java เนื่องจากสามารถแยกส่วนการประมวลผลจากการแสดงผลได้ดีขึ้น และมีการสนับสนุนการใช้งานที่เป็นมิตรต่อผู้พัฒนา เช่น การใช้ **Model-View-Controller (MVC)** ที่ช่วยให้การจัดการการแสดงผลและการประมวลผลข้อมูลทำได้ง่าย มีระเบียบ

3. JSP ใน Java EE (Jakarta EE)

ถึงแม้ว่า JSP จะไม่ได้รับความนิยมเหมือนเมื่อก่อน แต่ยังคงถูกใช้ในบางส่วนของ **Jakarta EE** (เดิมคือ **Java EE**) ซึ่งเป็นสเปคหลักสำหรับการพัฒนาเว็บแอปพลิเคชันใน Java องค์กรที่มีระบบที่ใช้ JSP อยู่แล้วก็ยังคงใช้ต่อไป แต่แนวโน้มการใช้งานลดลงเมื่อเทียบกับการใช้เทคโนโลยีที่ทันสมัยกว่า

4. การใช้งานในบางกรณี

JSP ยังเหมาะสมกับการใช้งานในโปรเจกต์ที่ต้องการโครงสร้างง่าย ๆ หรือไม่ต้องการฟังก์ชันการทำงานที่ซับซ้อนเกินไป ซึ่งมันยังคงให้ประโยชน์ในกรณีที่มีการสร้างเว็บแอปพลิเคชันแบบไดนามิกที่ไม่ซับซ้อนหรือใช้สำหรับการเรียนการสอนการพัฒนาเว็บแอปพลิเคชัน

5. การใช้งานในเครื่องมือเก่า

JSP ยังคงถูกใช้อยู่ในเครื่องมือหรือระบบที่ถูกพัฒนามาแล้วก่อนที่จะมีการแนะนำเทคโนโลยีใหม่ เช่น ในระบบ **Apache Tomcat** ซึ่งยังคงรองรับ JSP เพื่อความเข้ากันได้กับระบบเก่าและโปรเจกต์ที่ยังใช้ JSP อยู่

เทคโนโลยีที่มาแทนที่ JSP

- **Spring MVC:** ใช้แนวคิด **Model-View-Controller (MVC)** ซึ่งทำให้การแยกการประมวลผลและการแสดงผลข้อมูลเป็นระเบียบ และการทำงานกับ Spring Framework มีความยืดหยุ่นมากขึ้น
- **Thymeleaf:** เป็นเครื่องมือที่นิยมใช้ในการสร้างเทมเพลตสำหรับแสดงผลหน้าเว็บใน Spring Boot โดยให้การทำงานที่สะดวกและเป็นธรรมชาติมากขึ้น

อย่างไรก็ตาม JSP ยังสามารถใช้ได้ในบางกรณี โดยเฉพาะในโปรเจกต์ที่ต้องการความเรียบง่ายและไม่ได้ใช้เทคโนโลยีใหม่ๆ ที่ซับซ้อนมากนัก แต่ในภาพรวม, JSP ถูกแทนที่ด้วยเทคโนโลยีที่มีความยืดหยุ่นและสะดวกในการใช้งานมากกว่า เช่น Spring MVC และ Thymeleaf ซึ่งทำให้การพัฒนาเว็บแอปพลิเคชันในสภาพแวดล้อมของ Java มีความยืดหยุ่นและสะดวกมากขึ้น

การใช้งาน JSP (JavaServer Pages) ใน NetBeans IDE เป็นกระบวนการที่ไม่ซับซ้อน และสามารถทำได้ตามขั้นตอนที่แนะนำด้านล่างนี้ โดยใช้ Apache Tomcat เป็นเซิร์ฟเวอร์เพื่อรันแอปพลิเคชัน JSP

ส่วนประกอบของ JSP

1. **Directives:** ใช้ในการกำหนดพารามิเตอร์เกี่ยวกับการทำงานของ JSP เช่น การนำเข้าไลบรารีหรือการกำหนด encoding ของไฟล์
 - ตัวอย่าง:
 - `<%@ page language="java" contentType="text/html; charset=ISO-8859-1"%>`
2. **Declarations:** ใช้ในการประกาศตัวแปรหรือเมธอดที่สามารถใช้ได้ทั่วทั้งหน้า JSP
 - ตัวอย่าง:
 - `<%! int counter = 0; %>`
3. **Scriptlets:** ส่วนที่ฝังโค้ด Java ลงใน HTML ใช้สำหรับการเขียนคำสั่ง Java ที่จะถูกประมวลผลเมื่อหน้าเว็บถูกเรียกใช้งาน
 - ตัวอย่าง:
 - `<% out.println("Hello, world!"); %>`

4. **Expressions:** ใช้ในการแสดงค่าของตัวแปรหรือการคำนวณใน HTML
 - ตัวอย่าง:
 - `<%= 2 + 2 %> <!-- แสดงผลเป็น 4 -->`
5. **Actions:** ใช้ในการเรียกใช้คำสั่งหรือฟังก์ชันจากภายนอก เช่น การใช้งาน JavaBeans หรือการเชื่อมต่อกับฐานข้อมูล
 - ตัวอย่าง:
 - `<jsp:useBean id="user" class="com.example.User" />`
6. **Standard Tag Library (JSTL):** เป็นคอลเลกชันของแท็กที่ใช้ในการจัดการข้อมูล เช่น การวนลูปหรือการแสดงผลที่มีเงื่อนไข
 - ตัวอย่าง:
 - `<c:forEach var="item" items="${list}">`
 - `${item}`
 - `</c:forEach>`

ขั้นตอนการตั้งค่าและใช้งาน JSP ใน NetBeans

1. ติดตั้ง NetBeans IDE และ Apache Tomcat

- ดาวน์โหลดและติดตั้ง **NetBeans IDE** จากเว็บไซต์ [NetBeans](#)
- ดาวน์โหลดและติดตั้ง **Apache Tomcat** (เวอร์ชันที่รองรับ JSP และ Servlets) จากเว็บไซต์ [Tomcat](#)

2. ตั้งค่า Apache Tomcat ใน NetBeans

1. เปิด **NetBeans IDE** ขึ้นมา
2. ไปที่ **Tools > Servers > คลิกที่ Add Server...**
3. เลือก **Apache Tomcat** จากรายการเซิร์ฟเวอร์ที่รองรับ
4. เลือกตำแหน่งที่ติดตั้ง **Apache Tomcat** และกด **Next** ตามด้วยการตั้งค่าต่างๆ ที่จำเป็นจนเสร็จสิ้น

3. สร้างโปรเจกต์ JSP

1. ใน **NetBeans IDE**, ไปที่ **File > New Project...**
2. เลือก **Java Web > Web Application** แล้วกด **Next**
3. กำหนดชื่อโปรเจกต์และตำแหน่งการบันทึก
4. เลือก **Apache Tomcat** เป็นเซิร์ฟเวอร์ที่ใช้ในการรันโปรเจกต์
5. กด **Finish** เพื่อสร้างโปรเจกต์ใหม่

4. สร้างไฟล์ JSP

1. ใน **Project Explorer**, คลิกขวาที่ **Web Pages > เลือก New > JSP (JavaServer Page)**

2. กำหนดชื่อไฟล์ JSP เช่น index.jsp แล้วกด **Finish**

5. เขียนโค้ด JSP

- เปิดไฟล์ index.jsp แล้วเพิ่มโค้ดตัวอย่าง JSP ลงไป:
- `<%-- index.jsp --%>`
- `<html>`
- `<head>`
- `<title>Welcome to JSP</title>`
- `</head>`
- `<body>`
- `<h1>Welcome to JavaServer Pages (JSP)</h1>`
- `<%`
- `String message = "Hello, World!";`
- `out.println(message); // ใช้คำสั่ง Java เพื่อแสดงข้อความ`
- `%>`
- `</body>`
- `</html>`

6. รันโปรเจกต์

- คลิกขวาที่โปรเจกต์ใน **Project Explorer** และเลือก **Run** (หรือกด **F6**) เพื่อรันโปรเจกต์
- เว็บเบราว์เซอร์จะเปิดขึ้น และแสดงผลลัพธ์จาก index.jsp

7. ผลลัพธ์ในเว็บเบราว์เซอร์

- หน้าเว็บจะถูกแสดงผลตามโค้ดที่เขียนใน index.jsp โดยจะแสดงข้อความ **"Welcome to JavaServer Pages (JSP)"** และ **"Hello, World!"**
-

ตัวอย่างการใช้งาน JSP แบบละเอียด

สมมติว่าเราต้องการสร้างหน้าเว็บที่รับค่าจากผู้ใช้และแสดงผลข้อความจากแบบฟอร์ม:

1. สร้างไฟล์ JSP สำหรับรับค่าผู้ใช้:

- สร้างไฟล์ **form.jsp**:
- `<%-- form.jsp --%>`
- `<html>`
- `<head>`
- `<title>Form Input</title>`
- `</head>`
- `<body>`
- `<h1>Please Enter Your Name</h1>`

- `<form action="greeting.jsp" method="POST">`
- Name: `<input type="text" name="name" />`
- `<input type="submit" value="Submit" />`
- `</form>`
- `</body>`
- `</html>`

2. สร้างไฟล์ JSP สำหรับแสดงผล:

- สร้างไฟล์ **greeting.jsp**:
- `<!-- greeting.jsp -->`
- `<html>`
- `<head>`
- `<title>Greeting</title>`
- `</head>`
- `<body>`
- `<h1>Welcome,`
- `<%`
- `String name = request.getParameter("name"); // รับค่าจาก form`
- `if (name != null) {`
- `out.println(name); // แสดงชื่อที่กรอก`
- `} else {`
- `out.println("Guest");`
- `}`
- `%>!`
- `</h1>`
- `</body>`
- `</html>`

3. ทดสอบการทำงาน:

- รันโปรเจกต์ใน **NetBeans IDE** และเปิดหน้า form.jsp ผ่านเบราว์เซอร์
- กรอกชื่อในฟอร์มแล้วส่งข้อมูลไปยัง greeting.jsp ซึ่งจะทำการแสดงผลการต้อนรับตามชื่อที่กรอก

การใช้งาน **JSP (JavaServer Pages)** ใน **NetBeans IDE** และรันบน **GlassFish Web Server** มีขั้นตอนที่คล้ายกับการใช้งานใน Apache Tomcat เพียงแค่ต้องตั้งค่า **GlassFish** เป็น

เซิร์ฟเวอร์ที่ใช้งานในโปรเจกต์ ต่อไปนี้คือขั้นตอนและตัวอย่างการใช้งาน JSP บน GlassFish Web Server:

ขั้นตอนการตั้งค่าและใช้งาน JSP ใน NetBeans โดยใช้ GlassFish Web Server

1. ติดตั้ง NetBeans IDE และ GlassFish Server

- ดาวน์โหลดและติดตั้ง **NetBeans IDE** จากเว็บไซต์ [NetBeans](#)
- **GlassFish Server** มักจะมาพร้อมกับ **NetBeans IDE** อยู่แล้ว ถ้าไม่ได้ติดตั้ง GlassFish Server ให้ทำการดาวน์โหลดจากเว็บไซต์ [GlassFish](#) และติดตั้ง

2. ตั้งค่า GlassFish Server ใน NetBeans

1. เปิด **NetBeans IDE** ขึ้นมา
2. ไปที่ **Tools > Servers > คลิกที่ Add Server...**
3. เลือก **GlassFish Server** จากรายการเซิร์ฟเวอร์ที่รองรับ
4. กำหนดที่อยู่และตำแหน่งของการติดตั้ง **GlassFish Server** หากยังไม่ได้ติดตั้ง GlassFish ให้ทำการติดตั้งและระบุที่ตั้ง
5. กด **Next** แล้ว **Finish** หลังจากตั้งค่าเสร็จ

3. สร้างโปรเจกต์ JSP (Web Application) ใน NetBeans

1. ใน **NetBeans IDE**, ไปที่ **File > New Project...**
2. เลือก **Java Web > Web Application** แล้วกด **Next**
3. กำหนดชื่อโปรเจกต์และตำแหน่งการบันทึก
4. เลือก **GlassFish** เป็นเซิร์ฟเวอร์ที่ใช้ในการรันโปรเจกต์
5. กด **Finish** เพื่อสร้างโปรเจกต์ใหม่

4. สร้างไฟล์ JSP

1. ใน **Project Explorer**, คลิกขวาที่ **Web Pages > เลือก New > JSP (JavaServer Page)**
2. กำหนดชื่อไฟล์ JSP เช่น index.jsp แล้วกด **Finish**

5. เขียนโค้ด JSP

- เปิดไฟล์ index.jsp แล้วเพิ่มโค้ดตัวอย่าง JSP ลงไป:
- `<%-- index.jsp --%>`
- `<html>`
- `<head>`
- `<title>Welcome to JSP</title>`
- `</head>`
- `<body>`
- `<h1>Welcome to JavaServer Pages (JSP)</h1>`
- `<%`

- String message = "Hello, World!";
- out.println(message); // ใช้คำสั่ง Java เพื่อแสดงข้อความ
- %>
- </body>
- </html>

6. รันโปรเจกต์บน GlassFish Server

1. คลิกขวาที่โปรเจกต์ใน **Project Explorer** แล้วเลือก **Run** (หรือกด **F6**) เพื่อรันโปรเจกต์
2. **NetBeans** จะเริ่ม **GlassFish Server** และเปิดเบราว์เซอร์ที่แสดงหน้าเว็บจาก index.jsp
3. ถ้าไม่มีเบราว์เซอร์เปิดขึ้น, คุณสามารถเปิดเบราว์เซอร์เองและไปที่ <http://localhost:8080/your-project-name/index.jsp> เพื่อดูผลลัพธ์

ตัวอย่างการใช้งาน JSP ใน GlassFish

สมมติว่าต้องการสร้างหน้าเว็บที่รับค่าจากผู้ใช้และแสดงผลข้อความจากแบบฟอร์ม:

1. สร้างไฟล์ JSP สำหรับรับค่าผู้ใช้:
 - สร้างไฟล์ **form.jsp**:
 - <%-- form.jsp --%>
 - <html>
 - <head>
 - <title>Form Input</title>
 - </head>
 - <body>
 - <h1>Please Enter Your Name</h1>
 - <form action="greeting.jsp" method="POST">
 - Name: <input type="text" name="name" />
 - <input type="submit" value="Submit" />
 - </form>
 - </body>
 - </html>
2. สร้างไฟล์ JSP สำหรับแสดงผล:
 - สร้างไฟล์ **greeting.jsp**:
 - <%-- greeting.jsp --%>
 - <html>
 - <head>
 - <title>Greeting</title>

- `</head>`
- `<body>`
- `<h1>Welcome,`
- `<%`
- `String name = request.getParameter("name"); // รับค่าจาก form`
- `if (name != null) {`
- `out.println(name); // แสดงชื่อที่กรอก`
- `} else {`
- `out.println("Guest");`
- `}`
- `%>!`
- `</h1>`
- `</body>`
- `</html>`

3. ทดสอบการทำงาน:

- รันโปรเจกต์ใน **NetBeans IDE** และเปิดหน้า `form.jsp` ผ่านเบราว์เซอร์
- กรอกชื่อในฟอร์มแล้วส่งข้อมูลไปยัง `greeting.jsp` ซึ่งจะทำการแสดงผลการต้อนรับตามชื่อที่กรอก

รูปแบบการเขียนโค้ดของเจเอสพี

รูปแบบการเขียนภาษาเจเอสพี เขียนได้ในสามลักษณะดังนี้

1. สคริปต์เล็ต

เขียนภายใต้เครื่องหมาย `<% code %>` ซึ่งเป็นรูปแบบการเขียนหลักที่สามารถเปิดและปิดในโค้ดของภาษาเอชทีเอ็มแอลได้ตลอดเวลา ตัวอย่างแสดงดังนี้

```
<%
    int x=10;
    double 10.50;
    String s="Testing JSP String"
    int sum=x+100;
    out.println(sum);
%>
```

2. เดคลอเรชัน (Declarations)

เดคคลอเรชันเป็นการกำหนดเมธอดใช้งานด้วยโค้ดภาษาจาวาเพื่อเรียกใช้ในส่วนอื่นในหน้าเพจ มีรูปแบบที่ไป 2 แบบ คือ กำหนดด้วยรูปแบบเจเอสพี และแบบเอ็กซ์เอ็มแอลดังนี้

แบบเจเอสพี : `<%! code %>`

แบบเอ็กซ์เอ็มแอล : `<jsp:declaration> code </jsp:declaration>`

ตัวอย่างเช่น

```
<%!
    public String showName(String s){
        Return "สวัสดีคุณ."+s;
    }
%>
```

หรือ

```
<jsp:declaration>
    public String showName(String s){
        Return "สวัสดีคุณ."+s;
    }
</jsp:declaration>
```

3. เอ็กซ์เพรสชัน (Expressions)

การใช้งานแบบนี้ คือ ใช้งานคำสั่งแสดงตัวแปรที่ต้องการให้แสดงค่า ณ ตำแหน่งใดตำแหน่งหนึ่งในหน้าเพจ สามารถเขียนได้ทั้งรูปแบบของเจเอสพีและแบบเอ็กซ์เอ็มแอล ดังนี้

แบบเจเอสพี : `<%= code %>`

แบบเจเอสพี : `<jsp:expression> code </jsp:expression>`

```
<%=x%>
```

หรือ

```
<jsp:expression> x </jsp:expression>
```

ตัวอย่างหน้าเพจที่เขียนด้วยภาษาเจเอสพี

```
<%@page contentType="text/html" pageEncoding="UTF-8"%> //ส่วนหัวของเพจ
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
    "http://www.w3.org/TR/html4/loose.dtd">
<html>
    //โครงสร้างภาษา HTML
```

```
<head>           //ส่วนหัวของภาษา HTML
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
    <title>JSP Page</title> // ส่วนชื่อเรื่องของเพจ
</head>

<body>           //ส่วนการเริ่มต้น ของการเขียนโปรแกรม บอดี
<%
    int x=10;
    double 10.50;
    int sum=x+100;
    out.println(sum);
%>
<%!
    public String showName(String s){
        Return "สวัสดีคุณ:"+s;
    }
%>
<%=showName()>

</body>         //จบส่วนบอดี
</html>         //ส่วนการจบแฟ้ม HTML
```

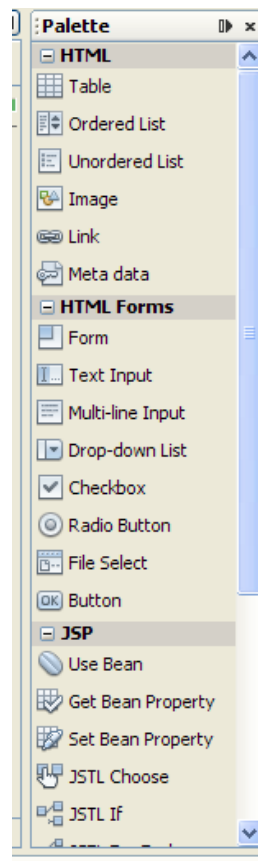
การใช้คำสั่งของภาษาเจเอสพี

1) คำสั่งแสดงผล

การใช้งานคำสั่งแสดงผลโดยทั่วไปจะใช้ out.println() เป็นหลัก โดยเขียนผ่านแท็ก เอ็กซ์เพรสชันเป็นต้น

2) คำสั่งรับข้อมูล

โดยทั่วไปจะไม่มีคำสั่งรับข้อมูลจากภาษาเจเอสพีโดยตรง แต่จะรับข้อมูลผ่านฟอร์มของภาษา HTML ซึ่งจะกล่าวในส่วนต่อไป แสดงดังภาพต่อไปนี้ โดยในการออกแบบโปรแกรมสามารถลากมาวางแล้วเขียนรหัสโปรแกรมตามแนวทางของการใช้งานโปรแกรมเนตบินได้ทันที ซึ่งจะแสดงในหัวข้อต่อไป



ภาพที่ 1.1 แสดงเครื่องมือสำหรับพัฒนาโปรแกรม

3) คำสั่งเงื่อนไข

คำสั่งประเภทนี้จะใช้คำสั่งของภาษาจาวาแทรกเข้าไปในภาษา HTML ได้ ทั้งคำสั่ง if และเงื่อนไขแบบ switch

ตัวอย่างการใช้งานเงื่อนไข if

```
<%
int x = 5;
if (x > 10) {
    out.println("OK.");
} else {
    out.println("Not OK.");
}
%>
```

ตัวอย่างการใช้งานเงื่อนไขแบบ switch

```
<%
int m = 5;
```

```
switch (m) {  
case 1 : {out.println("Banana"); break;}  
case 2 : {out.println("Apple"); break;}  
case 3 : {out.println("Lemon"); break;}  
case 4 : {out.println("Orange"); break;}  
default : {out.println("Should be 1-4");}  
}  
%>
```

คำสั่งวนซ้ำ

ตัวอย่างการใช้งานคำสั่ง for

```
<%  
for (int i=1;i<=100;i++){  
    out.println(i);  
}  
%>
```

ตัวอย่างการใช้งานลูป while

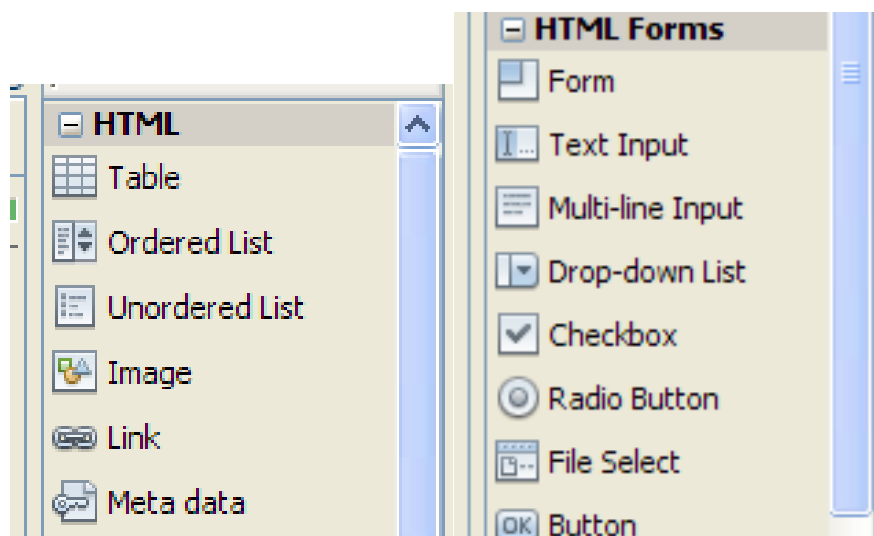
```
<%  
int i=0;  
while (i<= 10) {  
    out.println(i);  
    i++;  
}  
%>
```

ตัวอย่างการใช้งานลูป do...while

```
<%  
int i =0;  
do {  
    out.println(i);  
    i++;  
} while (i <= 100);  
%>
```

การใช้งานฟอร์มร่วมกับเจเอสพี

การใช้งานฟอร์มนั้นโดยทั่วไปมีวัตถุประสงค์ในการรับค่าจากผู้ใช้เพื่อประมวลผลอย่างใดอย่างหนึ่ง เช่น การส่งค่าเพื่อรายงานผล ส่งค่าเพื่อบันทึกลงฐานข้อมูล ส่งค่าฟอร์มจากภาษา HTML ไปยังฟอร์มตนเองหรือต่างฟอร์มก็ได้ การใช้งานฟอร์มจะใช้ส่วนดังภาพ

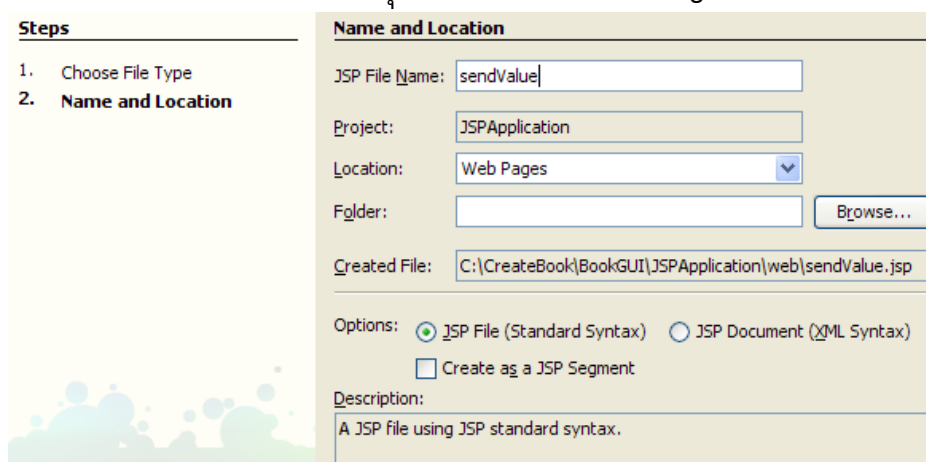


ภาพที่ 1.2 แสดงเครื่องมือจัดการ HTML และฟอร์ม

ตัวอย่างการใช้งานฟอร์มเพียงฟอร์มเดียวด้วยการส่งค่าให้กับฟอร์มของตนเอง เช่น การประกาศฟอร์มแล้วสร้างส่วนรับข้อมูลสองปุ่มแล้วรับข้อมูลจากผู้ใช้ หลังจากนั้น คลิกเพื่อส่งค่าแสดงให้กับฟอร์มตนเอง และแสดงผล โดยทำตามตัวอย่างต่อไปนี้

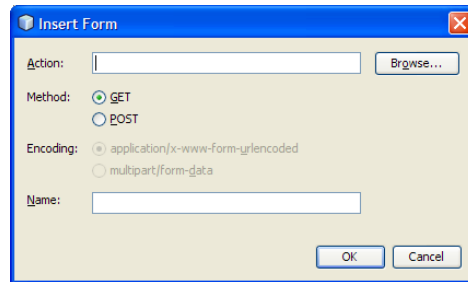
ตัวอย่างที่ 1.1 จงออกแบบฟอร์มรับชื่อและสกุล เมื่อคลิกปุ่มแสดงผล ให้แสดงชื่อและนามสกุลที่ป้อนให้แสดงบนเพจเดิม

1. สร้างแฟ้มเจเอสพี โดยการคลิกปุ่มขวาของเมาส์ที่ WebPage->JSP แล้วกำหนดชื่อ ดังภาพ



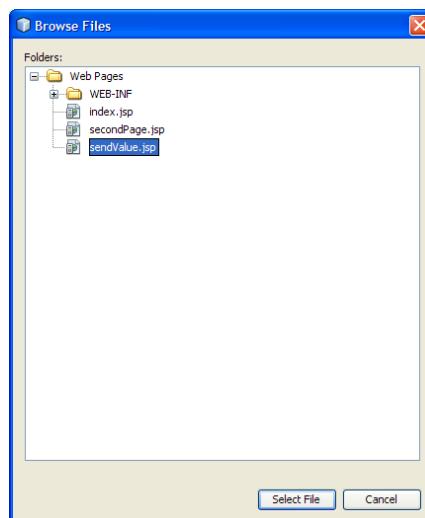
ภาพที่ 1.3 แสดงการสร้างเพจสำหรับส่งค่าข้อมูล

2. ออกแบบฟอร์มในส่วน <body> โดยการคลิกลาก Form มาวางดังภาพ



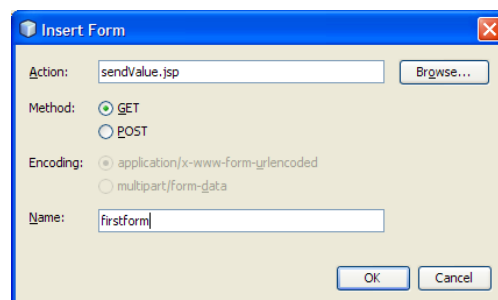
ภาพที่ 1.4 แสดงการสร้างฟอร์มเมื่อลากมาวาง

3. เลือก Browse เลือกชื่อตนเอง แล้วคลิก OK จะได้นี้



ภาพที่ 1.5 แสดงการเลือกฟอร์มที่จะเชื่อมโยง

4. Select File แล้วกำหนดชื่อฟอร์ม ดังภาพ

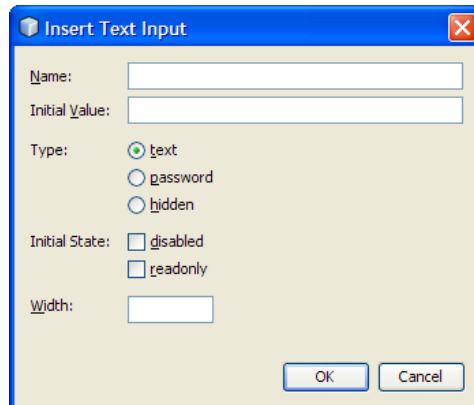


ภาพที่ 1.6 แสดงการกำหนดแอ็คชั่นของการเชื่อมโยง

5. คลิกปุ่ม OK จะได้รับรหัสโปรแกรมที่โปรแกรมเน็ตบีนสร้างให้ดังนี้

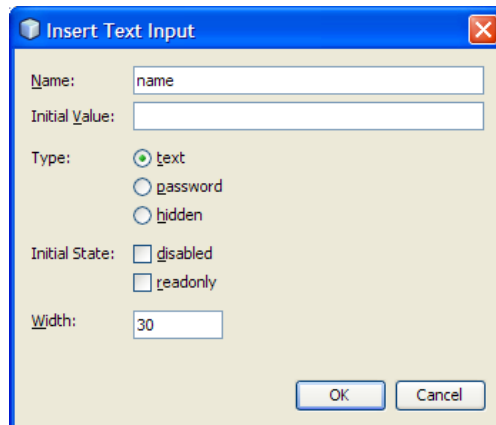
```
<form name="firstform" action="sendValue.jsp">
</form>
```

6. หลังจากนั้นคลิกที่ Text Input ลากมาวางภายในฟอร์ม (ระหว่าง <form> </form>) จะได้ดังนี้



ภาพที่ 1.7 แสดงกำหนดค่าการสร้างเท็กซ์ฟิลด์

7. กำหนดชื่อและความกว้างหรือกำหนดค่าเริ่มต้นด้วยก็ได้ ดังนี้



ภาพที่ 1.8 แสดงการกำหนดค่าเท็กซ์

8. จะได้รหัสโปรแกรมดังนี้

```
<input type="text" name="name" value="" size="30" />
```

9. ดำเนินการเช่นกันนั้นกับ นามสกุลจะได้รหัสโปรแกรมดังนี้

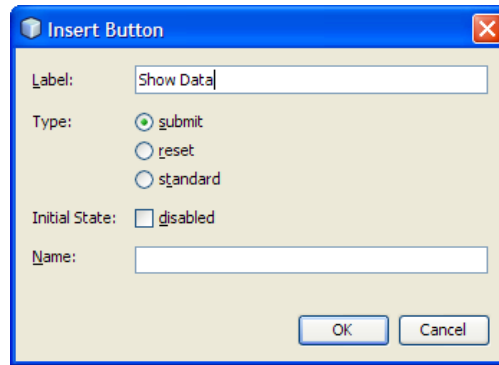
```
<input type="text" name="surname" value="" size="30" />
```

ให้เขียนข้อความและแท็ก
 เพื่อขึ้นบรรทัดใหม่เพิ่มต่อท้ายส่วนของ Text Input ทั้งสอง ซึ่งเมื่อสมบูรณ์แล้วจะได้ดังนี้

```
Name: <input type="text" name="name" value="" size="30" /><br>
```

```
Surname:<input type="text" name="surname" value="" size="30" /><br>
```

10. หลังจากลากส่วนของ Button มาวางเพื่อทำปุ่ม submit เพื่อแสดงผล เมื่อลากมาวางต่อท้ายส่วนของ Text Input จะได้ดังนี้



ภาพที่ 1.9 แสดงการกำหนดค่าของปุ่ม

11. ให้กำหนดเฉพาะส่วนของ Label ดังภาพด้านบน ซึ่งจะได้รหัสโปรแกรมดังนี้

```
<input type="submit" value="Show Data" />
```

12. หลังจากนั้นให้เขียนส่วนของเจเอสพีแท็ก หลังจาก </form> ดังนี้

```
<%
    String name=request.getParameter("name");
    String surname=request.getParameter("surname");
%>
<% out.println("Your Name:"+name); %> <br>
<% out.println("Your Surname:"+surname); %> <br>
```

รหัสโปรแกรมทั้งหมดจะได้ดังนี้

1. <%@page contentType="text/html" pageEncoding="UTF-8"%>
2. <!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
 "http://www.w3.org/TR/html4/loose.dtd">
3. <html>
4. <head>
5. <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
6. <title>JSP Page</title>
7. </head>
8. <body>
9. <h2>Show send Value in Single Page.</h2>
10. <form name="firstform" action="sendValue.jsp">
11. Name:<input type="text" name="name" value="" size="30" />

12. Surname:<input type="text" name="surname" value="" size="30" />

13. <input type="submit" value="Show Data" />
14. </form>

```

15.  <%
16.    String name=request.getParameter("name");
17.    String surname=request.getParameter("surname");
18.  %>
19.  <% out.println("Your Name:"+name); %> <br>
20.  <% out.println("Your Surname:"+surname); %> <br>
21. </body>
22. </html>

```

13. ผลการรันโปรแกรมจะได้ดังนี้

Show send Value in Single Page.

Name:

Surname:

Your Name:null

Your Surname:null

ภาพที่ 1.10 แสดงผลการรันของฟอร์มส่งค่า

เมื่อป้อนข้อมูลดังภาพ

Show send Value in Single Page.

Name:

Surname:

Your Name:null

Your Surname:null

ภาพที่ 1.11 แสดงการป้อนค่าก่อนการส่งค่า

หลังจากนั้นคลิกปุ่ม Show Data จะได้ดังนี้

Show send Value in Single Page.

Name:

Surname:

Your Name:Mr.BatMan

Your Surname:SuperHero

ภาพที่ 1.12 แสดงการส่งค่าเพจตนเอง

การส่งค่าข้ามฟอร์มก็จะมีลักษณะเช่นเดียวกันกับการส่งค่าในเพจ เพียงแต่เลือกแฟ้มที่ลิงค์เชื่อมโยงไปเป็นอีกแฟ้มหนึ่ง โดยสร้างฟอร์มไว้ที่แฟ้มหลักแล้ว หลังจากนั้นสร้างแฟ้มรับค่าที่จะส่งไป แล้วนำข้อมูลออกมาด้วย request.getParameter() ตามค่าที่ส่งเข้ามา ตัวอย่างต่อไปนี้เป็นการสร้างแฟ้มส่งค่าตัวเลขข้ามฟอร์มไปประมวลผลอีกฟอร์มหนึ่ง โดยฟอร์มแรกจะส่งค่าไปแล้วให้อีกฟอร์ม ที่รับค่านำไปคำนวณหาค่า ผลบวก ลบ คูณ และหาร แล้วแสดงผล

ตัวอย่างที่ 1.2 จงเขียนโปรแกรมส่งค่าข้ามเพจโดยเพจแรกเป็นตัวป้อนข้อมูล ส่งค่าให้เพจที่สอง นำไปแสดงผลบวก ลบ คูณ และหาร

1. สร้างฟอร์มป้อนข้อมูลดังนี้

Sending Data

Input Number 1:

Input Number 2:

ภาพที่ 1.13 แสดงการออกแบบก่อนการส่งค่าข้ามเพจ

รหัสโปรแกรมของแฟ้มนี้คือ

1. <%@page contentType="text/html" pageEncoding="UTF-8"%>
2. <!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN" "http://www.w3.org/TR/html4/loose.dtd">
3. <html>
4. <head>
5. <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
6. <title>JSP Page</title>
7. </head>
8. <body>
9. <h2>Sending Data</h2>
10. <form action="resultPage.jsp">
11. Input Number 1:<input type="text" name="numberone" value="" size="5" />

12. Input Number 2:<input type="text" name="numbertwo" value="" size="5" />

13. <input type="submit" value="Send Data" />
14. </form>
15. </body>
16. </html>

2. เขียนส่วนของเพจรับข้อมูล resultPage ดังนี้

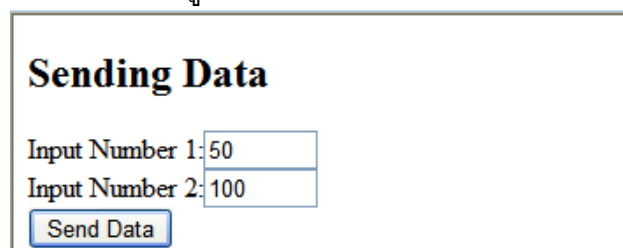
1. <%@page contentType="text/html" pageEncoding="UTF-8"%>

```

2. <!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
   "http://www.w3.org/TR/html4/loose.dtd">
3. <html>
4. <head>
5.   <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
6.   <title>JSP Page</title>
7. </head>
8. <body>
9.   <h2>Result Page</h2>
10.  <%
11.    String n1=request.getParameter("numberone");
12.    String n2=request.getParameter("numbertwo");
13.    double result1=Double.parseDouble(n1)+Double.parseDouble(n2);
14.    double result2=Double.parseDouble(n1)-Double.parseDouble(n2);
15.    double result3=Double.parseDouble(n1)*Double.parseDouble(n2);
16.    double result4=Double.parseDouble(n1)/Double.parseDouble(n2);
17.  %>
18.  <% out.println("Number 1:"+n1+" Number 2:"+n2);%> <br>
19.  <% out.println("Plus result:"+result1); %> <br>
20.  <% out.println("Minus result:"+result2); %> <br>
21.  <% out.println("Multiply result:"+result3); %> <br>
22.  <% out.println("Divide result:"+result4); %> <br>
23. </body>
24. </html>

```

3. ผลการรันเริ่มจากเพจรับข้อมูลดังนี้



ภาพที่ 1.14 แสดงการป้อนข้อมูลก่อนการส่งค่าข้ามเพจ

4. เมื่อป้อนข้อมูลแล้วคลิกปุ่ม Send Data จะให้ผลดังนี้

Result Page

Number 1:50 Number 2:100

Plus result:150.0

Minus result:-50.0

Multiply result:5000.0

Divide result:0.5

ภาพที่ 1.15 แสดงผลการส่งค่าข้ามเพจ

สรุป

JSP (Java Server Pages) คือเทคโนโลยีจาก Java สำหรับพัฒนาเว็บแอปพลิเคชันที่สร้างหน้าเว็บแบบ ใดนามิก ซึ่งทำงานร่วมกับ Java Servlets และ JavaBeans เพื่อจัดการข้อมูลและการประมวลผลฝั่งเซิร์ฟเวอร์ โดยเฉพาะอย่างยิ่งในการประมวลผลข้อมูลจากผู้ใช้ เช่น ฟอรมกรอกข้อมูล การเชื่อมต่อฐานข้อมูล หรือการแสดงผลที่เปลี่ยนแปลงตามเซิร์ฟเวอร์ ทั้งยังสามารถทำงานร่วมกับ Servlet ได้ โดย Servlet ทำหน้าที่ประมวลผลข้อมูลและ JSP ทำหน้าที่แสดงผล เทคโนโลยีนี้ถูกพัฒนาโดย Sun Microsystems (ปัจจุบันคือ Oracle Corporation) เพื่อแก้ไขปัญหาข้อจำกัดของ CGI และ SSI ในการสร้างเว็บแบบใดนามิกที่ซับซ้อน ทำให้การพัฒนาเว็บแอปพลิเคชันที่ใช้ Java มีความยืดหยุ่นและง่ายขึ้นด้วยการแยกการประมวลผลข้อมูลออกจากการแสดงผล โครงสร้างหลักของ JSP ซึ่งเป็นภาษาสคริปต์ที่ประมวลผลฝั่งเซิร์ฟเวอร์และขยายความสามารถของ Java Servlet นั้น ประกอบด้วย ไตเรกทีฟ (Directive) สำหรับกำหนดค่าการทำงาน เช่น page, include, taglib; แอ็คชั่น (Action) สำหรับจัดเตรียมและดำเนินการในหน้าเพจผ่านแท็กที่กำหนดไว้ล่วงหน้า เช่น <jsp:include>, <jsp:forward> หรือการเรียกใช้ JavaBean; สคริปต์เล็ต (Scriptlet) ซึ่งเป็นส่วนของการเขียนโค้ด Java ภายใน JSP; และ แท็กไลบรารี (Tag Libraries) สำหรับขยายความสามารถของภาษาโดยการกำหนดและเรียกใช้แท็กที่สร้างขึ้นเองได้